

ScrollSense: Assignment 1

c&d

Abhijeet Kumar
DA24B001

C: Logical design and relations

C.1 : Relation Listing

The following relations are mapped directly from the entity and relationship design in Deliverable B. I have kept the same entities and relationships and converted each one into a relation. Timestamps are shown with the logical domain `timestamp`; physical SQLite representation.

```
User(  
    user_id          : bigint          -- PK  
    auth_method      : text            -- CHECK IN ('phone','google')  
    phone_number      : text NULL      -- UNIQUE  
    google_account_id : text NULL      -- UNIQUE  
    email            : text NULL  
    display_name      : text  
    created_at        : timestamp  
)
```

PK: user_id

FK: none

```
UserHandlePeriod(  
    handle_period_id : bigint          -- PK  
    user_id          : bigint          -- FK -> User  
    handle           : text  
    valid_from       : timestamp  
    valid_to         : timestamp NULL  
)
```

PK: handle_period_id

FK: user_id -> User

```
UserStatusPeriod(  
    status_period_id : bigint          -- PK  
    user_id          : bigint          -- FK -> User  
    status           : text            -- CHECK IN  
                                         -- ('active','deactivated',  
                                         -- 'pending_deletion')  
    valid_from       : timestamp  
    valid_to         : timestamp NULL  
)
```

PK: status_period_id

FK: user_id -> User

```
InterestCategory(  
    category_id      : bigint          -- PK  
    category_name     : text            -- UNIQUE  
)
```

```

PK: category_id
FK: none

UserDeclaredInterest(
    user_id      : bigint      -- FK -> User
    category_id  : bigint      -- FK -> InterestCategory
    declared_at  : timestamp
)
PK: (user_id, category_id)
FK: user_id -> User
    category_id -> InterestCategory

UserInferredInterest(
    inferred_interest_id : bigint  -- PK
    user_id              : bigint  -- FK -> User
    category_id          : bigint  -- FK -> InterestCategory
    confidence_score     : real    -- CHECK BETWEEN 0 AND 1
    valid_from           : timestamp
    valid_to             : timestamp NULL
)
PK: inferred_interest_id
FK: user_id -> User
    category_id -> InterestCategory

InterestSuppression(
    suppression_id : bigint      -- PK
    user_id        : bigint      -- FK -> User
    category_id    : bigint      -- FK -> InterestCategory
    suppressed_from : timestamp
    suppressed_to   : timestamp NULL
)
PK: suppression_id
FK: user_id -> User
    category_id -> InterestCategory

Creator(
    creator_id      : bigint      -- PK
    user_id         : bigint      -- FK -> User, UNIQUE
    became_creator_at : timestamp
)
PK: creator_id
FK: user_id -> User

CreatorTierPeriod(
    tier_period_id : bigint      -- PK
    creator_id     : bigint      -- FK -> Creator
    tier            : text        -- CHECK IN (tier_enum)
    valid_from     : timestamp
    valid_to       : timestamp NULL
)
PK: tier_period_id
FK: creator_id -> Creator

Video(
    video_id      : bigint      -- PK
    owner_id      : bigint      -- FK -> User
    duration_ms   : integer      -- CHECK BETWEEN 20000 AND 90000
    caption       : text

```

```

        audio_track_id : bigint NULL      -- FK -> AudioTrack
        uploaded_at    : timestamp
    )
PK: video_id
FK: owner_id -> User
    audio_track_id -> AudioTrack

Hashtag(
    hashtag_id : bigint      -- PK
    tag_text    : text       -- UNIQUE, case-insensitive
)
PK: hashtag_id
FK: none

VideoHashtag(
    video_id      : bigint      -- FK -> Video
    hashtag_id    : bigint      -- FK -> Hashtag
)
PK: (video_id, hashtag_id)
FK: video_id -> Video
    hashtag_id -> Hashtag

AudioTrack(
    track_id      : bigint      -- PK
    origin_video_id : bigint NULL -- FK -> Video
    source_type    : text       -- CHECK IN ('original','licensed')
    title          : text
)
PK: track_id
FK: origin_video_id -> Video

ModerationDecision(
    decision_id    : bigint      -- PK
    video_id       : bigint      -- FK -> Video
    state          : text       -- CHECK IN
                                -- ('pending','live',
                                --  'age_restricted','demoted',
                                --  'taken_down')
    decided_by_type : text       -- CHECK IN ('classifier','human')
    reviewer_user_id : bigint NULL -- FK -> User
    classifier_version : text NULL
    decided_at     : timestamp
)
PK: decision_id
FK: video_id -> Video
    reviewer_user_id -> User

CHECK:
exactly one of reviewer_user_id and classifier_version is populated,
consistent with decided_by_type.

FollowPeriod(
    follow_period_id : bigint      -- PK
    follower_id      : bigint      -- FK -> User
    followee_id      : bigint      -- FK -> User
    started_at       : timestamp
    ended_at         : timestamp NULL
)

```

```

PK: follow_period_id
FK: follower_id -> User
   followee_id -> User

BlockPeriod(
    block_period_id : bigint          -- PK
    blocker_id      : bigint          -- FK -> User
    blocked_id      : bigint          -- FK -> User
    started_at      : timestamp
    ended_at        : timestamp NULL
)
PK: block_period_id
FK: blocker_id -> User
   blocked_id -> User

MutePeriod(
    mute_period_id : bigint          -- PK
    muter_id       : bigint          -- FK -> User
    muted_id       : bigint          -- FK -> User
    started_at     : timestamp
    ended_at       : timestamp NULL
)
PK: mute_period_id
FK: muter_id -> User
   muted_id -> User

Impression(
    impression_id   : bigint          -- PK
    user_id         : bigint          -- FK -> User
    video_id        : bigint          -- FK -> Video
    recommendation_id : bigint NULL   -- FK -> AgentRecommendation
    shown_at        : timestamp
    feed_position   : integer
    model_version   : text
)
PK: impression_id
FK: user_id -> User
   video_id -> Video
   recommendation_id -> AgentRecommendation

ViewSegment(
    view_segment_id : bigint          -- PK
    impression_id   : bigint          -- FK -> Impression
    segment_start_ms : integer
    segment_end_ms  : integer
    reached_end     : integer          -- CHECK IN (0,1)
)
PK: view_segment_id
FK: impression_id -> Impression

Signal(
    signal_id       : bigint          -- PK
    user_id         : bigint          -- FK -> User
    video_id        : bigint          -- FK -> Video
    signal_type     : text
    share_destination : text NULL
    comment_text    : text NULL
    retracts_signal_id : bigint NULL  -- FK -> Signal

```

```

        created_at          : timestamp
    )
    PK: signal_id
    FK: user_id -> User
        video_id -> Video
        retracts_signal_id -> Signal

CHECK:
signal_type IN
('like','save','share','comment','follow_from_feed',
 'not_interested','report','like_retraction')

Session(
    session_id : bigint          -- PK
    user_id    : bigint          -- FK -> User
    started_at : timestamp
)
PK: session_id
FK: user_id -> User

Turn(
    turn_id          : bigint -- PK
    session_id       : bigint -- FK -> Session
    turn_index       : integer
    user_message     : text
    assistant_message : text
    prompt_template_version_id : bigint -- FK -> PromptTemplateVersion
    model_id         : bigint -- FK -> Model
    temperature      : real
    input_tokens     : integer
    output_tokens    : integer
    cached_input_tokens : integer
    created_at       : timestamp
)
PK: turn_id
FK: session_id -> Session
    prompt_template_version_id -> PromptTemplateVersion
    model_id -> Model

ToolCall(
    tool_call_id      : bigint          -- PK
    turn_id           : bigint          -- FK -> Turn
    parent_tool_call_id : bigint NULL   -- FK -> ToolCall
    tool_name         : text
    arguments_json     : text
    result            : text
    latency_ms        : integer
    errored           : integer          -- CHECK IN (0,1)
    called_at         : timestamp
)
PK: tool_call_id
FK: turn_id -> Turn
    parent_tool_call_id -> ToolCall

PromptTemplate(
    template_id : bigint          -- PK
    template_name : text          -- UNIQUE
)

```

```

PK: template_id
FK: none

PromptTemplateVersion(
    template_version_id : bigint      -- PK
    template_id         : bigint      -- FK -> PromptTemplate
    version_number      : integer
    template_text       : text
    created_at          : timestamp
)
PK: template_version_id
FK: template_id -> PromptTemplate

Model(
    model_id   : bigint      -- PK
    model_name : text        -- UNIQUE
)
PK: model_id
FK: none

ModelPricingPeriod(
    pricing_period_id : bigint      -- PK
    model_id          : bigint      -- FK -> Model
    input_rate        : real
    output_rate       : real
    cached_input_rate : real
    valid_from        : timestamp
    valid_to          : timestamp NULL
)
PK: pricing_period_id
FK: model_id -> Model

AgentRecommendation(
    recommendation_id : bigint      -- PK
    turn_id           : bigint      -- FK -> Turn
    video_id          : bigint      -- FK -> Video
    shelf_position    : integer
    recommended_at    : timestamp
)
PK: recommendation_id
FK: turn_id -> Turn
    video_id -> Video

LLMJudgeScore(
    score_id   : bigint      -- PK
    turn_id    : bigint      -- FK -> Turn, UNIQUE
    helpfulness : real
    groundedness : real
    safety      : real
    judged_at   : timestamp
)
PK: score_id
FK: turn_id -> Turn

UserRating(
    rating_id : bigint      -- PK
    turn_id   : bigint      -- FK -> Turn, UNIQUE
    rating    : text        -- CHECK IN ('up','down')
)

```

```

    rated_at : timestamp
)
PK: rating_id
FK: turn_id -> Turn

```

There are 30 relations in total.

C.2 : Mapping Decision Log

For each decision, I mention the alternative that i rejected and the reason for rejecting it.

1. Video–Hashtag (M:N relationship).

A video can have multiple hashtags and the same hashtag can be used by many videos. Therefore, I mapped this M:N relationship to a separate **VideoHashtag** relation with the composite primary key (**video_id**, **hashtag_id**).

I rejected storing the hashtags as a comma-separated list inside **Video**. That would make individual hashtag searches and referential integrity difficult to enforce and would introduce repeating values inside one attribute.

2. User–InterestCategory for declared interests (M:N relationship).

A user can declare multiple interests, while an interest category can be declared by many users. I therefore created the associative relation **UserDeclaredInterest**, using (**user_id**, **category_id**) as its primary key.

I rejected adding columns such as **interest1**, **interest2**, and **interest3** to **User**. This would impose an artificial limit on the number of interests and would violate the relational approach to repeating values.

3. Weak entity ViewSegment.

A **ViewSegment** exists only in the context of an **Impression**, so it is mapped as a weak entity with **impression_id** as a foreign key.

I use a surrogate **view_segment_id** as its primary key rather than using (**impression_id**, **segment_sequence**). This keeps references to individual segments simple and avoids making every downstream reference depend on a composite key.

4. Optional single AudioTrack for Video.

The relationship from **Video** to **AudioTrack** is optional and at most one audio track is associated with a video. Therefore, **Video.audio_track_id** is implemented as a nullable foreign key.

I rejected a separate junction table because that would model the relationship as many-to-many and would not represent the cardinality in the ER model.

The reverse relationship, where an audio track can be sourced from an original video, is represented separately using **AudioTrack.origin_video_id**. This is why the two relations contain foreign keys in opposite directions: they represent two different relationships rather than one relationship being mapped twice.

5. Temporal relationships.

The relationships represented by **UserHandlePeriod**, **UserStatusPeriod**, **CreatorTierPeriod**, **FollowPeriod**, **BlockPeriod**, **MutePeriod**, and **ModelPricingPeriod** are mapped to their own relations containing **valid_from** and **valid_to**, or equivalent start/end timestamps.

I rejected storing only the current value on the parent entity. The main reason is the client's requirement to reconstruct what was knowable at a past timestamp. A current-value column would overwrite historical information and would therefore cause future information to leak into historical analysis.

6. ModerationDecision as an append-only history.

A video can have multiple moderation decisions over its lifetime, so every decision is stored as a separate row in **ModerationDecision**. The current state is obtained from the most recent decision rather than being stored separately in **Video**.

I rejected keeping both a **Video.current_state** column and the moderation history. That design would duplicate the same fact in two places and could produce inconsistent values if one write succeeds and the other fails.

7. Moderation decision attribution.

A moderation decision can be made either by a classifier or by a human reviewer. I therefore use `decided_by_type` together with nullable `reviewer_user_id` and `classifier_version`, with a `CHECK` constraint enforcing that the correct attribution is present.

I rejected creating separate human-review and classifier-decision relations. That would introduce a generalisation hierarchy that is not necessary for the queries required by the system and would make the moderation history harder to query as one timeline.

8. Self-referencing social relationships.

Follow, block, and mute are represented by separate temporal relations containing two foreign keys to `User`. For example, `follower_id` and `followee_id` both reference `User`.

I rejected storing these relationships as repeated columns on `User`, because the number of users followed, blocked, or muted is unbounded and the relationships also require their own historical timestamps.

9. Signal retraction.

A retraction is represented as a new `Signal` row with `retracts_signal_id` pointing to the original signal. This gives the database an explicit history of both the original action and its retraction.

I rejected an `is_retracted` flag on the original signal. A flag would only describe the latest state and would not record when the retraction occurred as a separate event.

10. Recursive ToolCall relationship.

Tool calls can be nested, so `ToolCall` contains the nullable self-referencing foreign key `parent_tool_call_id`. A `NULL` value represents a top-level tool call.

I rejected creating fixed columns such as `child_tool_1`, `child_tool_2`, etc. because the nesting depth and number of child calls are not fixed.

11. PromptTemplateVersion.

Turns reference a specific `PromptTemplateVersion`, rather than only the parent `PromptTemplate`. This preserves the exact prompt version used when a turn was generated.

I rejected storing only the current template ID in `Turn`. If the template were later updated, that design would make it impossible to determine which prompt text was actually used for an older turn.

12. Surrogate keys for event and temporal entities.

For most event and temporal relations I use a surrogate identifier such as `decision_id`, `impression_id`, or `pricing_period_id` as the primary key.

I rejected using large natural composite keys as the default. Composite keys would make foreign-key references longer and more cumbersome, especially for event tables that are referenced by other events. Business uniqueness can still be enforced separately where it is required.

C.3: Relational Algebra

F2 : Per-creator watch hours and completion rate

F2 has three important requirements: only the video's current moderation state may determine whether it is live; watch time must be aggregated across all view segments belonging to an impression; and every creator must remain in the result, including creators whose clips were never shown or watched.

First, the latest moderation decision for each video is identified.

```
LatestDecision =
  gamma_{video_id; MAX(decided_at) -> latest_at}
  (ModerationDecision)

CurrentState =
  LatestDecision
  join_{LatestDecision.video_id = ModerationDecision.video_id
```

```

        AND LatestDecision.latest_at = ModerationDecision.decided_at}
ModerationDecision

```

```

LiveVideo =
    sigma_{state = 'live'}(CurrentState)

```

Next, watch telemetry is reduced to one row per impression. This is necessary because one impression can contain several viewing segments.

```

ViewAgg =
    gamma_{impression_id;
        SUM(segment_end_ms - segment_start_ms) -> watch_ms,
        MAX(reached_end) -> completed}
    (ViewSegment)

```

The impression relation is then left-joined to this aggregate. This preserves an impression even when it has no view segment.

```

ImpressionStats =
    Impression
    left_outer_join_{Impression.impression_id = ViewAgg.impression_id}
    ViewAgg

```

At this stage, an impression with no view segment has no completion event and therefore contributes zero watch time and zero completion to the creator statistics. In the final SQL implementation this NULL-to-zero conversion is handled with COALESCE.

The live videos are then connected to their owners and impressions.

```

LiveImpressions =
    pi_{creator_id, impression_id, watch_ms, completed}
    (
        LiveVideo
        join_{LiveVideo.video_id = Video.video_id}
        Video
        join_{Video.owner_id = Creator.user_id}
        Creator
        join_{Video.video_id = ImpressionStats.video_id}
        ImpressionStats
    )

```

Finally, the data is aggregated per creator and left-joined to the complete creator relation.

```

CreatorAgg =
    gamma_{creator_id;
        SUM(watch_ms) -> total_watch_ms,
        AVG(completed) -> completion_rate}
    (LiveImpressions)

Result =
    Creator
    left_outer_join_{Creator.creator_id = CreatorAgg.creator_id}
    CreatorAgg

```

The final SQL uses COALESCE for creators with no qualifying impressions, so they appear with zero watch hours and zero completion rate rather than being dropped or displayed as NULL.

F3 : Videos with no audio track

Since `Video.audio_track_id` is nullable, the simplest relational algebra expression is:

```
Result =  
  sigma_{audio_track_id IS NULL}(Video)
```

This directly returns videos for which no audio track is assigned.

An equivalent anti-join formulation can also be written as:

```
VideosWithAudio =  
  pi_{video_id}  
  (  
    Video  
    join_{Video.audio_track_id = AudioTrack.track_id}  
    AudioTrack  
  )
```

```
NoAudioIds =  
  pi_{video_id}(Video) - VideosWithAudio
```

```
Result =  
  Video  
  join_{Video.video_id = NoAudioIds.video_id}  
  NoAudioIds
```

The second form is useful because the set difference operates only on the union-compatible `video_id` projections. It is also the relational algebra equivalent of an anti-join or `NOT EXISTS` query.

F12 : Sessions where a recommended clip was watched to completion

First, I reduce the view-segment relation to the impression IDs for which the end of the clip was reached.

```
CompletedImpressions =  
  pi_{impression_id}  
  (  
    sigma_{reached_end = 1}(ViewSegment)  
  )
```

The result is then connected through the agent recommendation chain:

```
Result =  
  pi_{session_id, turn_id, video_id, shelf_position}  
  (  
    Session  
    join_{Session.session_id = Turn.session_id}  
    Turn  
    join_{Turn.turn_id = AgentRecommendation.turn_id}  
    AgentRecommendation  
    join_{AgentRecommendation.recommendation_id =  
          Impression.recommendation_id}  
    Impression  
    join_{Impression.impression_id =  
          CompletedImpressions.impression_id}  
    CompletedImpressions  
  )
```

Projecting `CompletedImpressions` to only `impression_id` before the join is important. Otherwise, multiple view segments for the same impression could create duplicate result rows.

Equivalence-Preserving Selection Pushdown

For F2, consider the join between the current video state and **Video**.

Before the rewrite:

```
sigma_{state = 'live'}  
(  
  CurrentState  
  join_{CurrentState.video_id = Video.video_id}  
  Video  
)
```

The selection can be pushed below the join because the predicate `state = 'live'` uses only attributes of **CurrentState**.
After the rewrite:

```
(  
  sigma_{state = 'live'}(CurrentState)  
)  
join_{CurrentState.video_id = Video.video_id}  
Video
```

The two expressions are equivalent because filtering tuples of **CurrentState** before joining does not change which tuples can satisfy the join condition. The rewritten form is expected to be cheaper because non-live rows are removed before the join, reducing the number of tuples that the join has to process.

1 D. Normalization

For this deliverable, four relations are analyzed in detail. The selection includes the agent layer through the turn-cost analysis and temporal data through the historical model-pricing relation. The functional dependencies are stated from the business meaning of the data, and the candidate keys are established using attribute closures.

1.1 D.1 : Functional Dependencies, Closures, Candidate Keys and Normal Forms

1.1.1 TurnUsage

The task sheet uses the following flattened relation as the normalization example:

TurnUsage(*turn_id*, *model_id*, *priced_at*, *input_tokens*, *output_tokens*, *cached_tokens*, *input_rate*, *output_rate*, *cached_input_rate*)

This is a derived analysis relation, not an additional table in C.1. It combines the turn-level usage information with the historical pricing applicable at the time of the turn. The purpose is to test the normalization of the data before separating turn facts from pricing facts.

Functional Dependencies The following functional dependencies hold.

$$F_1 : \text{turn_id} \rightarrow \text{model_id}, \text{priced_at}, \text{input_tokens}, \text{output_tokens}, \text{cached_tokens}$$

This holds because one turn is associated with one model, one turn timestamp, and one recorded set of token counts. These values describe that specific turn.

$$F_2 : (\text{model_id}, \text{priced_at}) \rightarrow \text{input_rate}, \text{output_rate}, \text{cached_input_rate}$$

This holds because the applicable price for a model is determined by the historical pricing interval containing the pricing instant. Therefore, the same model at the same pricing instant receives the same rates.

Attribute Closures For $turn_id$:

$$\{turn_id\}^+ = \{turn_id, model_id, priced_at, input_tokens, output_tokens, cached_tokens, input_rate, output_rate, cached_input_rate\}$$

Using F_1 , we first obtain $model_id$, $priced_at$, and all turn-level token attributes. Since $model_id$ and $priced_at$ are now known, F_2 gives all three pricing attributes. Hence:

$$\{turn_id\}^+ = U$$

where U is the complete attribute set of TurnUsage.

Therefore, $turn_id$ is a superkey.

For $(model_id, priced_at)$:

$$\{model_id, priced_at\}^+ = \{model_id, priced_at, input_rate, output_rate, cached_input_rate\}$$

No dependency determines $turn_id$ or the turn-level token counts from this attribute set. Therefore:

$$\{model_id, priced_at\}^+ \neq U$$

and $(model_id, priced_at)$ is not a superkey of the complete relation.

Candidate Keys Since $turn_id$ determines all attributes and has no proper subset other than the empty set, whose closure does not determine the relation, it is a candidate key.

Therefore:

$$Candidatekey = \{turn_id\}$$

There is no second candidate key because $(model_id, priced_at)$ does not determine the complete relation.

Normal Form Verdict The dependency

$$(model_id, priced_at) \rightarrow (input_rate, output_rate, cached_input_rate)$$

is non-trivial, but $(model_id, priced_at)$ is not a superkey of TurnUsage.

Therefore, TurnUsage violates BCNF.

The relation is decomposed into:

$$R_1 = TurnUsage(turn_id, model_id, priced_at, input_tokens, output_tokens, cached_tokens)$$

and

$$R_2 = ModelPricingAtTime(model_id, priced_at, input_rate, output_rate, cached_input_rate)$$

The common attributes are:

$$R_1 \cap R_2 = \{model_id, priced_at\}$$

The intersection is a candidate key of R_2 . Therefore, the decomposition is lossless.

The decomposition also preserves the stated dependencies: F_1 is represented in R_1 , while F_2 is represented in R_2 .

In the actual ScrollSense design, the second relation corresponds to the historical pricing stored in **ModelPricingPeriod**, with $priced_at$ matched against the interval $[valid_from, valid_to)$. This prevents a later pricing change from altering the historical cost of an earlier turn.

1.1.2 ModelPricingPeriod

The relation is:

$$ModelPricingPeriod(pricing_period_id, model_id, input_rate, output_rate, cached_input_rate, valid_from, valid_to)$$

This is a temporal relation because it stores pricing history rather than only the currently applicable price.

Functional Dependencies The primary key gives:

$$F_1 : \text{pricing_period_id} \rightarrow \text{model_id}, \text{input_rate}, \text{output_rate}, \text{cached_input_rate}, \text{valid_from}, \text{valid_to}$$

This holds because each pricing period has one unique identifier.

The historical pricing business rule is that, for a given model, a pricing period is uniquely identified by its start timestamp. Hence:

$$F_2 : (\text{model_id}, \text{valid_from}) \rightarrow \text{pricing_period_id}, \text{input_rate}, \text{output_rate}, \text{cached_input_rate}, \text{valid_to}$$

This means that two pricing periods for the same model cannot have the same *valid_from*.

Attribute Closures For *pricing_period_id*:

$$\{\text{pricing_period_id}\}^+ = \{\text{pricing_period_id}, \text{model_id}, \text{input_rate}, \text{output_rate}, \text{cached_input_rate}, \text{valid_from}, \text{valid_to}\}$$

Therefore, *pricing_period_id* is a superkey.

For *(model_id, valid_from)*:

$$\{\text{model_id}, \text{valid_from}\}^+ = \{\text{model_id}, \text{valid_from}, \text{pricing_period_id}, \text{input_rate}, \text{output_rate}, \text{cached_input_rate}, \text{valid_to}\}$$

Therefore, *(model_id, valid_from)* is also a superkey.

Candidate Keys Both determinants are minimal:

$$\{\text{pricing_period_id}\}$$

and

$$\{\text{model_id}, \text{valid_from}\}$$

are candidate keys.

Normal Form Verdict The left-hand side of every non-trivial FD is a candidate key and therefore a superkey.

Hence no non-trivial FD violates BCNF.

$$\text{ModelPricingPeriod is in BCNF}$$

1.1.3 PromptTemplateVersion

The relation is:

$$\text{PromptTemplateVersion}(\text{template_version_id}, \text{template_id}, \text{version_number}, \text{template_text}, \text{created_at})$$

The relation stores an immutable version of a prompt template. A version number is unique within a particular prompt template.

Functional Dependencies The primary key gives:

$$F_1 : \text{template_version_id} \rightarrow \text{template_id}, \text{version_number}, \text{template_text}, \text{created_at}$$

This holds because each version has one unique identifier.

The versioning rule gives:

$$F_2 : (\text{template_id}, \text{version_number}) \rightarrow \text{template_version_id}, \text{template_text}, \text{created_at}$$

This holds because version numbers are assigned uniquely within each prompt template: template version 1, version 2, version 3, and so on.

Attribute Closures For $template_version_id$:

$$\{template_version_id\}^+ = \{template_version_id, template_id, version_number, template_text, created_at\}$$

Therefore, $template_version_id$ is a superkey.

For $(template_id, version_number)$:

$$\{template_id, version_number\}^+ = \{template_id, version_number, template_version_id, template_text, created_at\}$$

Therefore, $(template_id, version_number)$ is also a superkey.

Candidate Keys Both determinants are minimal candidate keys:

$$\{template_version_id\}$$

and

$$\{template_id, version_number\}$$

Normal Form Verdict Every non-trivial FD has a candidate key as its determinant. Therefore:

$$PromptTemplateVersionis in BCNF$$

1.1.4 Creator

The relation is:

$$Creator(creator_id, user_id, became_creator_at)$$

The logical design specifies $user_id$ as unique in the Creator relation. Therefore, a user can correspond to at most one creator identity.

Functional Dependencies The primary key gives:

$$F_1 : creator_id \rightarrow user_id, became_creator_at$$

Because $user_id$ is unique in Creator:

$$F_2 : user_id \rightarrow creator_id, became_creator_at$$

The second dependency is therefore a real business constraint represented directly by the logical design.

Attribute Closures For $creator_id$:

$$\{creator_id\}^+ = \{creator_id, user_id, became_creator_at\}$$

Therefore, $creator_id$ is a superkey.

For $user_id$:

$$\{user_id\}^+ = \{user_id, creator_id, became_creator_at\}$$

Therefore, $user_id$ is also a superkey.

Candidate Keys Both determinants are minimal, so the candidate keys are:

$$\{creator_id\}$$

and

$$\{user_id\}$$

Normal Form Verdict Every non-trivial FD has a candidate key on its left-hand side. Therefore:

Creator is in BCNF

Summary

The normalization results are:

Relation	Candidate Keys	Normal Form
TurnUsage	<i>turn_id</i>	BCNF after decomposition
ModelPricingPeriod	<i>pricing_period_id</i> , (<i>model_id</i> , <i>valid_from</i>)	BCNF
PromptTemplateVersion	<i>template_version_id</i> , (<i>template_id</i> , <i>version_number</i>)	BCNF
Creator	<i>creator_id</i> , <i>user_id</i>	BCNF

Thus, three of the four analyzed relations are already in BCNF. TurnUsage is not in BCNF because pricing attributes are determined by (*model_id*, *priced_at*), which is not a superkey of the complete flattened relation. The lossless decomposition separates turn-specific usage facts from historical pricing facts.

D.2 : The Denormalization Not Done

A denormalization i considered was storing a derived **like_count** directly in the **Video** relation.

In the normalized design, the number of likes is derived from the **Signal** relation. For example, suppose Video 101 has the following signals:

User	Video	Signal Type
U1	V101	like
U2	V101	like
U3	V101	like
U4	V101	save

Only three of these signals are likes, so the current like count for Video 101 is:

$$like_count = 3$$

Instead of calculating this from **Signal** every time, I could store:

Video(*video_id*, ..., *like_count*)

The main advantage would be query performance. For example, a “top videos by likes” query could directly sort the stored counter:

```
SELECT video_id, like_count
FROM Video
ORDER BY like_count DESC;
```

Without the denormalized column, the database has to derive the count from the **Signal** history by filtering for like events and aggregating them.

However, this introduces an update anomaly because the same fact is now represented in two places: the **Signal** history and the stored **Video.like_count**.

For example, suppose Video 101 currently has:

$$Video.like_count = 3$$

A fourth user then likes the video. The system must perform both of the following actions:

1. insert a new **Signal** row with **signal_type** = 'like';
2. increase **Video.like_count** from 3 to 4.

If the `Signal` row is inserted successfully but the counter is not updated, the database becomes inconsistent:

`Signal history = 4likes`

but

`Video.like_count = 3`

The same problem occurs for a like retraction. In `ScrollSense`, a retraction is stored as a separate `like_retraction` `Signal` event. Therefore, the counter would have to be decremented when such an event is recorded.

For example:

3likes → newlike → 4likes → likeretract → 3likes

If either counter update is missed, the denormalized value can drift from the underlying event history.

To keep the counter consistent, the database would require an enforcement mechanism such as a trigger on `Signal`. An insertion of a `like` would increment `Video.like_count`, while an insertion of a `like_retraction` would decrement it. A reconciliation query could also be used periodically to compare the stored count with the value derived from `Signal`.

I deliberately do not implement this denormalization. The `Signal` relation remains the authoritative source of interaction history, so there is only one source of truth. Although storing `like_count` could make leaderboard queries faster, the additional consistency-maintenance mechanism would introduce unnecessary complexity for the current design.